

負載平衡器的 Cloud NGFW Essentials

來源：<https://codelabs.developers.google.com/lb-fw-ga?hl=zh-tw>

步驟數：10

目錄

1. [1. 簡介](#)
2. [2. 概念](#)
3. [3. 專案設定](#)
4. [4. 網路基礎](#)
5. [5. 負載平衡器服務](#)
6. [6. 用戶端存取權](#)
7. [7. 負載平衡器防火牆](#)
8. [8. 防火牆規則記錄](#)
9. [9. 清除所用資源](#)
10. [10. 結論](#)

1. 簡介

本程式碼實驗室將探討如何使用[區域網路防火牆政策](#)，為內部應用程式負載平衡器 (ALB) 和 Proxy 網路負載平衡器 (NLB) 設定 Cloud Next Generation Firewall (NGFW) *Essentials*。

Cloud NGFW 是完全分散式防火牆服務，提供進階威脅防護和微區隔功能，可保護 Google Cloud 工作負載。在負載平衡器層級啟用 Cloud NGFW，可將一致的防火牆政策規則套用至任何進入內部 Proxy 型負載平衡器的 *TCP* 流量。這個系統會針對所有服務更廣泛地強制執行政策，簡化機構資安態勢的佈建作業。

本程式碼研究室涵蓋下列 Cloud NGFW 和 Cloud Load Balancing 產品與功能：

- Cloud NGFW *Essentials*
- 區域性防火牆政策
- 區域性內部應用程式負載平衡器
- 後端代管執行個體群組 (MIG) 和 Private Service Connect (PSC) 網路端點群組 (NEG)

注意：如要瞭解負載平衡器[目標](#)的防火牆政策規則，請參閱 Cloud NGFW 說明文件，瞭解最新支援的功能和限制。

課程內容

- 啟用以負載平衡器為目標的基本 Cloud NGFW 防火牆政策規則
- 使用 VM 執行個體和 PSC 後端保護內部消費者負載平衡器服務
- 測試用戶端存取權及驗證防火牆記錄

需求條件

- 具備 Google Cloud 專案
 - 熟悉 Google Cloud 網路概念，並會使用 Google Cloud CLI
 - IAM 權限：`roles/compute.instanceAdmin.v1`、`roles/compute.networkAdmin`、`roles/compute.securityAdmin` 和 `roles/storage.admin`
-

2. 概念

防火牆功能層級

Cloud NGFW 提供三種功能層級：Essentials、Standard 和 Enterprise。每個進階層級都提供額外的網路流量篩選和檢查功能。

Cloud NGFW *Essentials* 篩選功能摘要：

級別	功能	網路層	規則參數範例
Essentials	IP 位址和範圍篩選	IP	<code>--src-ip-ranges=1.1.1.0/24,2.2.2.2/32</code>
Essentials	位址群組	IP	<code>--src-address-groups=special-ranges</code>
Essentials	通訊協定和通訊埠篩選	TCP	<code>--layer4-configs=tcp</code>
Essentials	安全標記	中繼資料	<code>--src-secure-tags=tagValues/987654321098</code>
Essentials	依網路類型篩選	IP / 中繼資料	<code>--src-network-type=INTRA_VPC</code>

負載平衡器轉送規則會明確定義目的地 TCP 通訊埠。防火牆規則 `--layer4-configs=` 參數只能指定 `tcp`。通訊埠值是由轉送規則本身所隱含。

位址群組和網路類型有助於提高防火牆政策規則的效率。負載平衡器的防火牆政策規則支援 `VPC_NETWORKS` 和 `INTRA_VPC` 網路類型。

注意：負載平衡器的防火牆政策規則僅支援 `--direction=INGRESS`。這些規則旨在控管負載平衡器公開的服務存取權。

注意：本程式碼研究室涵蓋負載平衡器的 Cloud NGFW *Essentials* 防火牆政策規則。負載平衡器的 Cloud NFGW *Enterprise* 防火牆政策規則實作方式，請參閱其他程式碼研究室。

資料層篩選

Cloud NFGW *Essentials* 功能涵蓋基本的第 3 層 (IP 位址) 和第 4 層 (TCP 通訊埠) 具狀態防火牆規則。這些防火牆政策規則功能都會在負載平衡器資料平面中有效執行，不需要完整檢查封包。

Cloud NGFW *Essentials* 政策規則會以 **VM 執行個體** 為目標，並在分散式虛擬私有雲網路架構中強制執行，這是 Google Cloud 軟體定義網路 (*Andromeda*) 的一部分。封包抵達 VM 執行個體網路介面時，系統會在每個 VM 執行個體的 Hypervisor 層級強制執行封包篩選和防火牆政策規則。

系統會使用 [Google Cloud Load Balancer 的基礎技術](#) (具體來說是 Envoy 服務 Proxy 基礎架構)，強制執行以負載平衡器為目標的 Cloud NGFW *Essentials* 政策規則。使用相同的 Cloud NFGW 資源模型和規則結構，直接在以 Proxy 為基礎的負載平衡器資料平面中強制執行有狀態封包篩選。

負載平衡器目標

以負載平衡器為目標的 Cloud NGFW 政策，與以虛擬機器執行個體為目標的政策之間，有幾項主要差異。

如要將防火牆政策規則套用至單一負載平衡器，請指定 `--target-type=INTERNAL_MANAGED_LB`，並一併指定負載平衡器轉送規則的特定參照 `--target-forwarding-rules=FR_NAME`。如要以虛擬私有雲網路區域中的所有負載平衡器轉送規則為目標 (政策會限定區域)，請省略特定參照，只需要 `--target-type=INTERNAL_MANAGED_LB` 旗標。

如果規則設定中未設定 `--target-type` 參數，則規則預設會自動套用至所有 VM 執行個體，**不會** 套用至負載平衡器。

注意：負載平衡器的預設防火牆政策規則行為為 `action=allow`。如果內部負載平衡器「未」附加防火牆政策，這是正常的日常行為。不過，如果封包不符合任何已定義的規則，這項行為也適用於已附加防火牆政策的負載平衡器。如要變更這項預設行為，請設定優先順序較低 (優先順序數字較高) 的明確「拒絕所有要求」規則，做為適用所有要求的規則。

程式碼研究室網路

本程式碼研究室使用單一專案，其中包含一個虛擬私有雲網路和下列資源：

- 兩個區域子網路
- 一項區域網路防火牆政策
- 三個區域性內部應用程式負載平衡器
 - `www` 具備 VM 執行個體群組後端的 HTTP 服務
 - `api` 具備 VM 執行個體群組後端的 HTTP 服務
 - `gcs` 搭配 PSC NEG 後端的 HTTPS 服務，可連線至 Google API
- 兩個 VM 執行個體，用於測試各種允許和拒絕政策

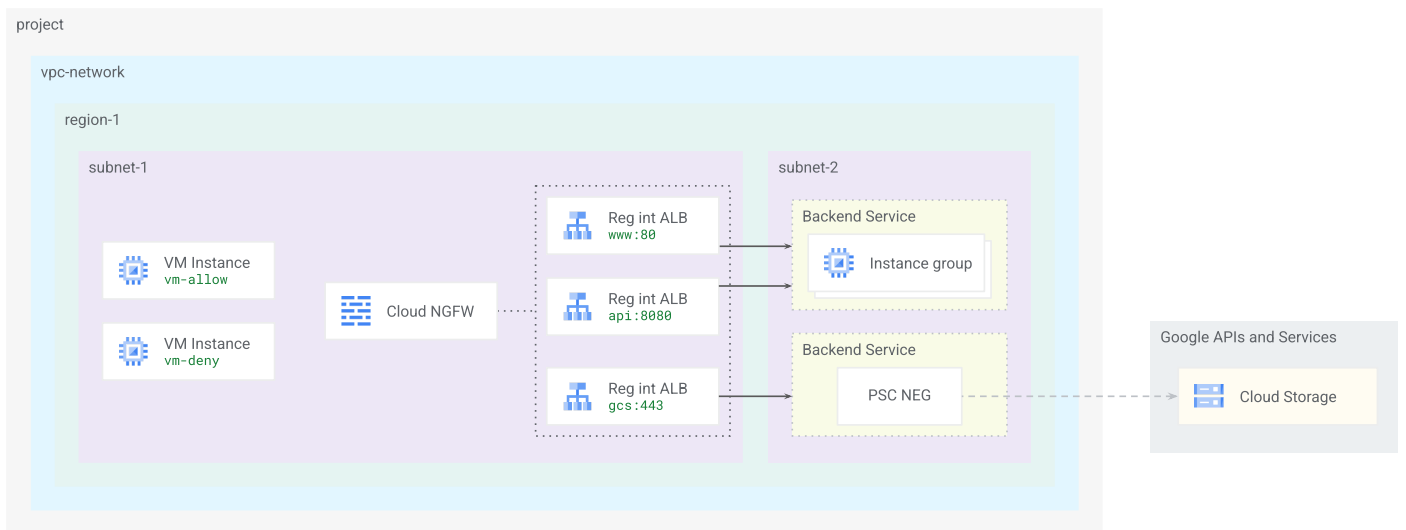


圖 1. 程式碼研究室網路

以負載平衡器為目標的防火牆政策規則會連結至負載平衡器轉送規則資源。負載平衡器本身是由個別定義的資源組成，這些資源會一起設定，提供完整的負載平衡服務。轉送規則定義會直接參照為其定義的特定目標 Proxy 資源。

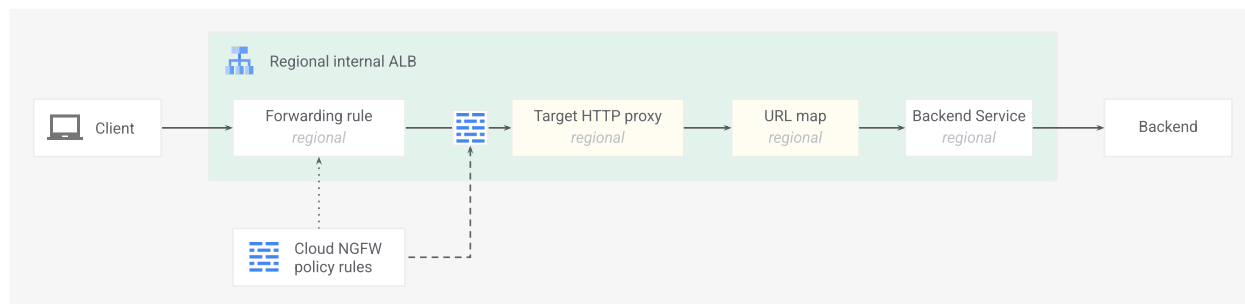


圖 1. 負載平衡器資源的 *Cloud NGFW*

Cloud NGFW *Essentials* 篩選器會編程至負載平衡器資料層，並在定義的目標 Proxy 服務層 (類似於 VM 執行個體介面) 實作，使用相同的分散式一致性防火牆機制來強制執行政策。

步驟 3 · 約 5 分鐘

3. 專案設定

存取專案

本程式碼研究室使用單一 Google Cloud 專案。設定步驟會使用 `gcloud cli` CLI 和 Linux 殼層指令。

首先，請存取 Google Cloud 雲端專案指令列：

- Cloud Shell (shell.cloud.google.com) 或
- 已安裝 `gcloud` CLI 的本機終端機

設定專案 ID

```
gcloud config set project YOUR_PROJECT_ID_HERE
```

啟用 API 服務

```
gcloud services enable \  
  cloudresourcemanager.googleapis.com \  
  compute.googleapis.com \  
  dns.googleapis.com \  
  networksecurity.googleapis.com \  
  certificatemanager.googleapis.com
```

設定殼層環境變數

```
# set your region preference  
export REGION_1="us-west1"
```

```
# set your zone preference  
export ZONE_1="us-west1-c"
```

```
# fetch project info and verify vars set  
export PROJECT_ID=$(gcloud config list --format="value(core.project)")  
export PROJECT_NO=$(gcloud projects describe ${PROJECT_ID} --format="value(projectNumber)")  
echo ${REGION_1}  
echo ${ZONE_1}  
echo ${PROJECT_ID}  
echo ${PROJECT_NO}
```

注意：搶先版功能可能會使用 `gcloud beta` 指令。如未安裝，請執行 `gcloud components install beta` 啟用。

4. 網路基礎

在本節中，您將部署網路基礎架構，包括：

- 全域虛擬私有雲網路和區域子網路
- 區域網路防火牆政策，可保護虛擬私有雲網路
- 伺服器專用的 Cloud Router 和 Cloud NAT，可擷取軟體套件
- 負載平衡器 Ingress 的 IP 位址預留和 DNS 記錄

建立網路資源

```
# create vpc network
gcloud compute networks create vnet-foo --subnet-mode=custom
```

```
# create subnet for clients
gcloud compute networks subnets create subnet-foo-1 \
  --network=vnet-foo \
  --region=${REGION_1} \
  --range=10.0.0.0/24 \
  --enable-private-ip-google-access
```

```
# create subnet for backend servers
gcloud compute networks subnets create subnet-foo-2 \
  --network=vnet-foo \
  --region=${REGION_1} \
  --range=172.16.0.0/24 \
  --enable-private-ip-google-access
```

```
# create proxy subnet
gcloud compute networks subnets create subnet-foo-3 \
  --purpose=REGIONAL_MANAGED_PROXY \
  --role=ACTIVE \
  --network=vnet-foo \
  --region=${REGION_1} \
  --range=172.16.128.0/23
```

建立防火牆元件

稍後新增負載平衡器專屬目標時，會用到這裡建立的基本區域網路防火牆政策。政策必須與負載平衡器位於相同區域。

建立位址群組

首先，請建立位址群組，找出支援負載平衡器功能的來源健康狀態檢查[探測 IP 範圍](#)。必須允許這些範圍，負載平衡器後端才能視為健康。稍後，這項標記也會用於以負載平衡器為目標的防火牆政策規則。

```
# create address group
gcloud network-security address-groups create uhc-probes \
  --description="health check probes" \
  --type=IPv4 \
  --capacity=42 \
  --location=${REGION_1}
```

```
# add ip ranges to address group
gcloud network-security address-groups add-items uhc-probes \
  --items=35.191.0.0/16,130.211.0.0/22 \
  --location=${REGION_1}
```

建立防火牆政策

```
# create fw policy
gcloud compute network-firewall-policies create fw-policy-foo-${REGION_1} \
  --description="foo fw ${REGION_1}" \
  --region=${REGION_1}
```

```
# create fw policy rule to allow in iap
gcloud compute network-firewall-policies rules create 1001 \
  --description="allow iap for ssh" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp:22 \
  --src-ip-ranges=35.235.240.0/20
```

```
# create fw policy rule to allow in health checks
gcloud compute network-firewall-policies rules create 1002 \
  --description="allow health checks to backends" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-address-groups=projects/${PROJECT_ID}/locations/${REGION_1}/addressGroups/uhc-probes
```

```
# create fw policy rule to allow in lb proxies
gcloud compute network-firewall-policies rules create 1003 \
  --description="allow lb proxy" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp:80,tcp:443,tcp:8080 \
  --src-ip-ranges=172.16.128.0/23
```

```
# associate fw policy to vnet
gcloud compute network-firewall-policies associations create \
  --name=fw-policy-association-foo-${REGION_1} \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --network=vnet-foo \
  --firewall-policy-region=${REGION_1}
```

設定網路服務

建立 Cloud Router 和 NAT 閘道

```
# create router for nat
gcloud compute routers create cr-nat-foo \
  --network=vnet-foo \
  --asn=16550 \
  --region=${REGION_1}
```

```
# create nat gateway
gcloud compute routers nats create natgw-foo \
  --router=cr-nat-foo \
  --region=${REGION_1} \
  --auto-allocate-nat-external-ips \
  --nat-all-subnet-ip-ranges
```

保留 IP 位址

```
# reserve vip for lb www service
gcloud compute addresses create vip-foo-www \
  --region=${REGION_1} \
  --subnet=subnet-foo-1 \
  --addresses=10.0.0.101
```

```
# reserve vip for lb api service
gcloud compute addresses create vip-foo-api \
  --region=${REGION_1} \
  --subnet=subnet-foo-1 \
  --addresses=10.0.0.102
```

```
# reserve vip for lb gcs service
gcloud compute addresses create vip-foo-gcs \
  --region=${REGION_1} \
  --subnet=subnet-foo-1 \
  --addresses=10.0.0.103
```

建立 DNS 記錄

```
# create dns zone
gcloud dns managed-zones create zone-foo \
  --description="private zone for foo" \
  --dns-name=foo.com \
  --networks=vnet-foo \
  --visibility=private
```

```
# create dns record for www service
gcloud dns record-sets create www.foo.com \
  --zone=zone-foo \
  --type=A \
  --ttl=300 \
  --rrdatas="10.0.0.101"
```

```
# create dns record for api service
gcloud dns record-sets create api.foo.com \
  --zone=zone-foo \
  --type=A \
  --ttl=300 \
  --rrdatas="10.0.0.102"
```

```
# create dns record for gcs service
gcloud dns record-sets create gcs.foo.com \
  --zone=zone-foo \
  --type=A \
  --ttl=300 \
  --rrdatas="10.0.0.103"
```

網路設定部分到此結束，接下來要設定負載平衡器。

5. 負載平衡器服務

在本節中，您將為下列三項服務部署負載平衡器元件 (後端服務、網址對應、目標 Proxy 和轉送規則)：

1. 通訊埠 80 上的 `www` 服務 (`ilb-foo-www`)
2. 通訊埠 8080 上的 `api` 服務 (`ilb-foo-api`)
3. 通訊埠 443 上的 `gcs` 服務 (`ilb-foo-gcs`)，並使用傳輸層安全標準 (TLS) 憑證

以及支援的後端資源：

1. 代管執行個體群組中執行 HTTP 伺服器的 VM 執行個體
2. Private Service Connect (PSC) 網路端點群組 (NEG) 至 Google API
3. Google Cloud Storage (GCS) bucket

設定後端資源

建立 VM 執行個體群組伺服器

`www` 負載平衡器會使用 VM 執行個體群組後端伺服器，執行接聽通訊埠 80 的 Apache 網路伺服器。

`api` 負載平衡器會使用同一個 VM 執行個體群組，並接聽通訊埠 8080。

```
# create vm startup config with http server
cat > vm-server-startup.sh << 'EOF'
#!/bin/bash
set -e
apt-get update
apt-get install apache2 -y
vm_hostname="$(curl -H "Metadata-Flavor:Google" \
http://169.254.169.254/computeMetadata/v1/instance/name)"
vm_zone="$(curl -H "Metadata-Flavor:Google" \
http://169.254.169.254/computeMetadata/v1/instance/zone | cut -d/ -f4)"
echo "www served from: $vm_hostname in zone $vm_zone on port 80" | \
tee /var/www/html/index.html
echo "Listen 8080" | tee -a /etc/apache2/ports.conf
mkdir -p /var/www/api
echo "api served from: $vm_hostname in zone $vm_zone on port 8080" | \
tee /var/www/api/index.html
tee /etc/apache2/sites-available/api.conf << EOF
<VirtualHost *:8080>
    DocumentRoot /var/www/api
</VirtualHost>
EOF
a2ensite api.conf
systemctl restart apache2
EOF
```

```
# create managed instance group template
gcloud compute instance-templates create mig-template-foo \
  --machine-type=e2-micro \
  --network=vnet-foo \
  --region=${REGION_1} \
  --subnet=subnet-foo-2 \
  --no-address \
  --shielded-secure-boot \
  --metadata-from-file=startup-script=vm-server-startup.sh
```

```
# create regional managed instance group
gcloud compute instance-groups managed create mig-foo \
  --region=${REGION_1} \
  --size=2 \
  --template=mig-template-foo \
  --base-instance-name=service-foo
```

```
# create named ports for instance group
gcloud compute instance-groups managed set-named-ports mig-foo \
  --named-ports=www-port:80,api-port:8080 \
  --region=${REGION_1}
```

建立 Storage bucket

gcs 負載平衡器會使用 PSC NEG 後端，透過 Google API 前端連線至 Cloud Storage 值區。

```
# create random bucket name
export BUCKET=$(openssl rand -hex 12)
echo ${BUCKET}
```

注意：關閉殼層工作階段後，環境變數就會遺失。如有需要，請記下 bucket 名稱，以便在日後的工作階段中完成作業。

```
# create bucket
gcloud storage buckets create gs://${BUCKET} --location=${REGION_1}
```

```
# give compute sa object admin role on bucket
gcloud storage buckets add-iam-policy-binding gs://${BUCKET} \
  --member=serviceAccount:${PROJECT_NO}-compute@developer.gserviceaccount.com \
  --role=roles/storage.objectAdmin
```

建立憑證

gcs 負載平衡器會使用部署至目標 HTTPS Proxy 的自簽憑證，終止用戶端 HTTPS 要求。

```
# create cert
openssl req -x509 -newkey rsa:2048 \
  -nodes \
  -days 365 \
  -keyout foo-gcs-key.pem \
  -out foo-gcs-cert.pem \
  -subj "/CN=Foo, Inc." \
  -addext "subjectAltName=DNS:gcs.foo.com"
```

```
# upload to certificate manager
gcloud certificate-manager certificates create cert-foo-gcs \
  --private-key-file=foo-gcs-key.pem \
  --certificate-file=foo-gcs-cert.pem \
  --location=${REGION_1}
```

建立負載平衡器元件

請使用下列指令碼，自動部署負載平衡器元件。這有助於提高所有相關設定元素的處理速度和準確度。

部署負載平衡器建立指令碼

```
# create script file
cat > create_lbs.sh << EOF
#!/bin/bash
set -e

# --- Create load balancer for www service port 80 ---
echo "--- Creating Load Balancer for WWW Service (ilb-foo-www) on port 80 ---"

echo "ilb-foo-www: creating health check (hc-foo-www)"
gcloud compute health-checks create http hc-foo-www \
  --use-serving-port \
  --region=${REGION_1}

echo "ilb-foo-www: creating backend service (bes-foo-www)"
gcloud compute backend-services create bes-foo-www \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --protocol=HTTP \
  --port-name=www-port \
  --health-checks=hc-foo-www \
  --health-checks-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-www: adding managed instance group (mig-foo) to backend service (bes-foo-www)"
gcloud compute backend-services add-backend bes-foo-www \
  --balancing-mode=UTILIZATION \
  --instance-group=mig-foo \
  --instance-group-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-www: creating url map (ilb-foo-www)"
gcloud compute url-maps create ilb-foo-www \
  --default-service=bes-foo-www \
  --region=${REGION_1}

echo "ilb-foo-www: creating target http proxy (proxy-foo-www)"
gcloud compute target-http-proxies create proxy-foo-www \
  --url-map=ilb-foo-www \
  --url-map-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-www: creating forwarding rule (fr-foo-www)"
gcloud compute forwarding-rules create fr-foo-www \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --network=vnet-foo \
  --subnet=subnet-foo-1 \
  --subnet-region=${REGION_1} \
  --address=vip-foo-www \
  --ports=80 \
  --target-http-proxy=proxy-foo-www \
  --target-http-proxy-region=${REGION_1} \
  --region=${REGION_1}
```

```
echo "--- Successfully created Load Balancer for WWW Service (ilb-foo-www) ---"
echo

# --- Create load balancer for api service port 8080 ---
echo "--- Creating Load Balancer for API Service (ilb-foo-api) on port 8080 ---"

echo "ilb-foo-api: creating health check (hc-foo-api)"
gcloud compute health-checks create http hc-foo-api \
  --use-serving-port \
  --region=${REGION_1}

echo "ilb-foo-api: creating backend service (bes-foo-api)"
gcloud compute backend-services create bes-foo-api \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --protocol=HTTP \
  --port-name=api-port \
  --health-checks=hc-foo-api \
  --health-checks-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-api: adding managed instance group (mig-foo) to backend service (bes-foo-api)"
gcloud compute backend-services add-backend bes-foo-api \
  --balancing-mode=UTILIZATION \
  --instance-group=mig-foo \
  --instance-group-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-api: creating url map (ilb-foo-api)"
gcloud compute url-maps create ilb-foo-api \
  --default-service=bes-foo-api \
  --region=${REGION_1}

echo "ilb-foo-api: creating target http proxy (proxy-foo-api)"
gcloud compute target-http-proxies create proxy-foo-api \
  --url-map=ilb-foo-api \
  --url-map-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-api: creating forwarding rule (fr-foo-api)"
gcloud compute forwarding-rules create fr-foo-api \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --network=vnet-foo \
  --subnet=subnet-foo-1 \
  --subnet-region=${REGION_1} \
  --address=vip-foo-api \
  --ports=8080 \
  --target-http-proxy=proxy-foo-api \
  --target-http-proxy-region=${REGION_1} \
  --region=${REGION_1}

echo "--- Successfully created Load Balancer for API Service (ilb-foo-api) ---"
echo
```

```

# --- Create load balancer for gcs service port 443 ---
echo "--- Creating Load Balancer for GCS Service (ilb-foo-gcs) on port 443 ---"

echo "ilb-foo-gcs: creating network endpoint group (neg-psc-gcs)"
gcloud compute network-endpoint-groups create neg-psc-gcs \
  --network-endpoint-type=private-service-connect \
  --psc-target-service=storage.${REGION_1}.rep.googleapis.com \
  --region=${REGION_1}

echo "ilb-foo-gcs: creating backend service (bes-foo-gcs)"
gcloud compute backend-services create bes-foo-gcs \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --protocol=HTTPS \
  --region=${REGION_1}

echo "ilb-foo-gcs: adding network endpoint group (neg-psc-gcs) to backend service (bes-foo-gcs)"
gcloud compute backend-services add-backend bes-foo-gcs \
  --network-endpoint-group=neg-psc-gcs \
  --network-endpoint-group-region=${REGION_1} \
  --region=${REGION_1}

echo "ilb-foo-gcs: creating url map (ilb-foo-gcs)"
gcloud compute url-maps create ilb-foo-gcs \
  --default-service=bes-foo-gcs \
  --region=${REGION_1}

echo "ilb-foo-gcs: creating target https proxy (proxy-foo-gcs)"
gcloud compute target-https-proxies create proxy-foo-gcs \
  --url-map=ilb-foo-gcs \
  --url-map-region=${REGION_1} \
  --certificate-manager-certificates=cert-foo-gcs \
  --region=${REGION_1}

echo "ilb-foo-gcs: creating forwarding rule (fr-foo-gcs)"
gcloud compute forwarding-rules create fr-foo-gcs \
  --load-balancing-scheme=INTERNAL_MANAGED \
  --network=vnet-foo \
  --subnet=subnet-foo-1 \
  --subnet-region=${REGION_1} \
  --address=vip-foo-gcs \
  --ports=443 \
  --target-https-proxy=proxy-foo-gcs \
  --target-https-proxy-region=${REGION_1} \
  --region=${REGION_1}

echo "--- Successfully created Load Balancer for GCS Service (ilb-foo-gcs) ---"
echo

echo "All load balancers created successfully."
EOF

```

```
# make script executable
chmod +x create_lbs.sh
```

```
# run script
./create_lbs.sh
```

注意：這個指令碼需要幾分鐘才能完成。

驗證負載平衡器建立作業

確認已部署轉送規則和後端服務。

```
# check forwarding rules
gcloud compute forwarding-rules list
```

```
# check backend services
gcloud compute backend-services list
```

負載平衡器設定部分到此結束，接下來要設定用戶端 VM 執行個體。

6. 用戶端存取權

建立 VM 用戶端資源

在本節中，您將部署用戶端並驗證端對端連線。

建立 VM 執行個體

```
# set variables for client ip addresses
export VM_ALLOW_IP="10.0.0.11"
export VM_DENY_IP="10.0.0.12"
echo ${VM_ALLOW_IP}
echo ${VM_DENY_IP}
```

```
# create client 1 vm
gcloud compute instances create vm-allow \
  --machine-type=e2-micro \
  --zone=${ZONE_1} \
  --subnet=subnet-foo-1 \
  --no-address \
  --private-network-ip=${VM_ALLOW_IP} \
  --scopes=cloud-platform \
  --shielded-secure-boot
```

```
# create client 2 vm
gcloud compute instances create vm-deny \
  --machine-type=e2-micro \
  --zone=${ZONE_1} \
  --subnet=subnet-foo-1 \
  --no-address \
  --private-network-ip=${VM_DENY_IP} \
  --scopes=cloud-platform \
  --shielded-secure-boot
```

測試基準服務

用戶端 `vm-allow` 的測試

注意：發出 `instances create` 指令後不久，VM 執行個體就會上線，並可透過 `ssh` 使用 IAP 存取。如果第一次要求失敗，可能需要稍候片刻。

```
# send request to foo www service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  curl -s www.foo.com"
```

```
# send request to foo api service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  curl -s api.foo.com:8080"
```

檢查是否收到成功回應：

```
www served from: service-foo-{uid} in zone {zone} on port 80
api served from: service-foo-{uid} in zone {zone} on port 8080
```

測試透過負載平衡器將檔案上傳至 Google Cloud Storage。

```
# send request to foo gcs service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  echo 'test one on the way' > test-upload-1.txt
  TOKEN=$(gcloud auth print-access-token)
  curl -s -k -X POST "https://gcs.foo.com/upload/storage/v1/b/${BUCKET}/o?
uploadType=media&name=test-upload-object-1.txt\" \
  -H \"Authorization: Bearer ${TOKEN}\" \
  -H \"Content-Type: text/plain\" \
  --data-binary @test-upload-1.txt"
```

檢查是否已成功確認：

```
"kind": "storage#object",
"id": "${BUCKET}/test-upload-object-1.txt/{GENID}",
```

Cloud Storage API 回應會確認網路路徑是否正常運作。

用戶端 `vm-deny` 的測試

```
# send request to foo www service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  curl -s www.foo.com"
```

```
# send request to foo api service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  curl -s api.foo.com:8080"
```

```
# send request to foo gcs service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  echo 'test two on the way' > test-upload-2.txt
  TOKEN=$(gcloud auth print-access-token)
  curl -s -k -X POST \"https://gcs.foo.com/upload/storage/v1/b/${BUCKET}/o?
uploadType=media&name=test-upload-object-2.txt\" \
  -H \"Authorization: Bearer \${TOKEN}\" \
  -H \"Content-Type: text/plain\" \
  --data-binary @test-upload-2.txt"
```

注意：這些測試也應該會成功，因為目前沒有防火牆規則以負載平衡器為目標。

主要設定部分到此結束，接下來要建立負載平衡器防火牆規則。

7. 負載平衡器防火牆

在本節中，您將部署以負載平衡器為目標的防火牆政策規則。這一連串的設定將建構資安態勢，允許存取 `vm-allow`，並封鎖所有服務的 `vm-deny` 流量。

允許特定流量前往 `fr-foo-www`

在現有防火牆政策 `fw-policy-foo-${REGION_1}` 中新增防火牆政策規則

- 允許包含 `vm-allow` 但排除 `vm-deny` IP 位址的來源 IP 範圍
- 新增其他來源篩選器 `INTRA_VPC`，在以負載平衡器為目標的防火牆政策規則中，使用網路類型

搭配其他來源參數使用時，防火牆政策規則會支援以負載平衡器為目標的 `INTRA_VPC` 和 `VPC_NETWORKS` 來源網路類型。評估邏輯是兩個來源參數之間的 `AND`。此時流量必須符合 `INTRA_VPC` 和 `--src-ip-ranges=${VM_ALLOW_IP}/32` 的條件，才能獲得許可。

建立規則以允許 `vm-allow` 指定目標 `fr-foo-www`

```
# create fw policy rule
gcloud beta compute network-firewall-policies rules create 2001 \
  --description="allow vm traffic to fr-www" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --enable-logging \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-network-type=INTRA_VPC \
  --src-ip-ranges=${VM_ALLOW_IP}/32 \
  --target-type=INTERNAL_MANAGED_LB \
  --target-forwarding-rules=projects/${PROJECT_ID}/regions/${REGION_1}/forwardingRules/fr-foo-www
```

用戶端 `vm-allow` 的測試

```
# send request to foo www service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  curl -s www.foo.com"
```

檢查是否收到成功的回應：

```
www served from: service-foo-{uid} in zone {zone} on port 80
```

用戶端 `vm-deny` 的測試

```
# send request to foo www service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  curl -s www.foo.com"
```

檢查是否收到成功的回應：

```
www served from: service-foo-{uid} in zone {zone} on port 80
```

注意：這項作業會成功，是因為負載平衡器的隱含預設防火牆政策規則行為是 `--action=allow`。如要變更這項設定，必須使用預設 (*catchall*) 拒絕規則。

拒絕 `fr-foo-www` 的預設流量

新增優先順序較低 (優先順序數字較高) 的防火牆政策規則。

- 拒絕來自任何來源 IP 位址的所有流量
- 在觸發拒絕規則前，系統會允許從 `vm-allow` 到 `fr-foo-www` 的流量

建立規則，拒絕指定目標的流量 `fr-foo-www`

```
# create fw policy rule
gcloud beta compute network-firewall-policies rules create 2999 \
  --description="allow vm traffic to fr-www" \
  --firewall-policy=fr-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --enable-logging \
  --action=deny \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-ip-ranges=0.0.0.0/0 \
  --target-type=INTERNAL_MANAGED_LB \
  --target-forwarding-rules=projects/${PROJECT_ID}/regions/${REGION_1}/forwardingRules/fr-foo-www
```

健康狀態檢查注意事項

與以 *VM 執行個體* 為目標的防火牆政策規則相同，預設的萬用 (隱含) 拒絕輸入規則會封鎖來自健康狀態檢查探測範圍的流量，這些流量的目的地是負載平衡器後端。因此，我們設定了明確的允許規則，允許輸入健康狀態檢查探測範圍 (請參閱規則 1002)。

重要事項：同樣地，為負載平衡器目標建立萬用 (明確) 拒絕輸入規則時，必須建立另一個優先順序較高 (優先順序數字較小) 的規則，允許來自健康狀態檢查探測器範圍的輸入流量。這項規則需要以負載平衡器為目標。

```
# create fw policy rule
gcloud beta compute network-firewall-policies rules create 2002 \
  --description="allow health checks to fr-www" \
  --firewall-policy=fr-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-address-groups=projects/${PROJECT_ID}/locations/${REGION_1}/addressGroups/uhc-probes \
  --target-type=INTERNAL_MANAGED_LB \
  --target-forwarding-rules=projects/${PROJECT_ID}/regions/${REGION_1}/forwardingRules/fr-foo-www
```

用戶端 `vm-deny` 的測試

```
# send request to foo www service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  curl -s www.foo.com"
```

現在應該會失敗，因為防火牆規則 `2999` 會拒絕來自虛擬私有雲網路的所有流量。優先順序較高 (優先順序數字較小) 的規則 `2001` 只允許包含 `vm-allow` 的來源範圍。

檢查失敗的回應： ...

按下 `Ctrl+C` 鍵，停止 `curl` 程序。

```
# send request to foo api service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  curl -s api.foo.com:8080"
```

檢查是否收到成功的回應：

```
api served from: service-foo-{uid} in zone {zone} on port 8080
```

`vm-deny` 仍可存取 API 服務！之所以成功，是因為防火牆規則只專門套用至轉送規則 `fr-foo-www`，並未指定 `fr-foo-api`。

更新規則，將所有負載平衡器設為目標

注意：如要將防火牆政策規則套用至虛擬私有雲網路中的**所有**負載平衡器，請省略 `--target-forwarding-rules=FR_NAME`。

變更防火牆政策規則，現在會套用至虛擬私有雲網路中的所有負載平衡器轉送規則目標。

1. 建立新的輸入「允許」規則 `2003`，將所有轉送規則設為目標，允許 VM 流量 (`vm-allow` IP 範圍)
2. 建立新的輸入 `allow` 規則 `2004`，以所有轉送規則為目標，允許健康狀態檢查 (`uhc-probes` 位址群組) 流量。

3. 建立新的輸入 *deny* 規則 2998 將所有轉送規則設為目標，做為所有其他流量的全部拒絕規則

修改防火牆規則，以所有負載平衡器為目標

```
# create fw policy rule
gcloud beta compute network-firewall-policies rules create 2003 \
  --description="allow vm traffic to all vnet lb fr" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --enable-logging \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-ip-ranges=${VM_ALLOW_IP}/32 \
  --target-type=INTERNAL_MANAGED_LB
```

```
# create fw policy rule
gcloud beta compute network-firewall-policies rules create 2004 \
  --description="allow health checks to all vnet lb fr" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --enable-logging \
  --action=allow \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-address-groups=projects/${PROJECT_ID}/locations/${REGION_1}/addressGroups/uhc-probes \
  --target-type=INTERNAL_MANAGED_LB
```

```
# create fw policy rule
gcloud beta compute network-firewall-policies rules create 2998 \
  --description="deny all vnet traffic to all vnet lb fr" \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1} \
  --enable-logging \
  --action=deny \
  --direction=INGRESS \
  --layer4-configs=tcp \
  --src-ip-ranges=0.0.0.0/0 \
  --target-type=INTERNAL_MANAGED_LB
```

由於現在針對虛擬私有雲網路中所有轉送規則的規則已是多餘，因此可以移除先前針對*明確*負載平衡器轉送規則的防火牆政策規則。

```
# delete redundant fw policy rules
gcloud beta compute network-firewall-policies rules delete 2001 \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1}

gcloud beta compute network-firewall-policies rules delete 2002 \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1}

gcloud beta compute network-firewall-policies rules delete 2999 \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1}
```

用戶端 `vm-deny` 的測試

```
# send request to foo api service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  curl -s api.foo.com:8080"
```

現在應該會失敗，因為所有防火牆政策規則也以 `--target-type=INTERNAL_MANAGED_LB` 為目標。 `fr-foo-api`

檢查失敗的回應：

...

按下 `Ctrl+C` 鍵，停止 `curl` 程序。

測試透過負載平衡器從 Google Cloud Storage 下載檔案。

```
# send request to foo gcs service
gcloud compute ssh vm-deny --zone=${ZONE_1} --command="
  TOKEN=$(gcloud auth print-access-token)
  curl -s -k \"https://gcs.foo.com/storage/v1/b/${BUCKET}/o/test-upload-object.txt?
  alt=media\" \
  -H \"Authorization: Bearer ${TOKEN}\" \
  -o test-download.txt"
```

按下 `Ctrl+C` 鍵，停止 `curl` 程序。

用戶端 `vm-allow` 的測試

```
# send request to foo www service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  curl -s www.foo.com"
```

```
# send request to foo api service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  curl -s api.foo.com:8080"
```

檢查是否收到成功回應：

```
www served from: service-foo-{uid} in zone {zone} on port 80
api served from: service-foo-{uid} in zone {zone} on port 8080
```

```
# send request to foo gcs service
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  TOKEN=$(gcloud auth print-access-token)
  curl -s -k \"https://gcs.foo.com/storage/v1/b/${BUCKET}/o/test-upload-object-1.txt?
  alt=media\" \
  -H \"Authorization: Bearer ${TOKEN}\" \
  -o test-download-1.txt"
```

驗證下載內容

```
# send request from vm
gcloud compute ssh vm-allow --zone=${ZONE_1} --command="
  cat test-download-1.txt"
```

檢查輸出是否成功：

```
test one on the way
```

`vm-allow` 可以使用所有負載平衡器服務，`vm-deny` 則已成功遭到封鎖。

測試部分到此結束，接下來我們將簡要介紹記錄功能。

8. 防火牆規則記錄

[防火牆記錄格式](#) 包含以負載平衡器為目標的規則 (`--target-type=INTERNAL_MANAGED_LB`) 欄位和記錄。

記錄檔會包含標示為 `load_balancer_details` 的額外欄位，其中提供防火牆政策規則的目標負載平衡器相關資訊。這與在防火牆政策規則中指定 VM 執行個體時的 `InstanceDetails` 欄位格式類似。

- `load_balancer_details.forwarding_rule_name` 顯示防火牆政策規則的目標轉送規則
- `load_balancer_details.type` 表示目標為哪種 Proxy 型負載平衡器
- 如果類型是應用程式負載平衡器，`load_balancer_details.url_map_name` 會記錄使用的網址對應資源

查看記錄

查詢防火牆記錄，查看防火牆政策規則的結果。

```
gcloud logging read \
  "logName=projects/${PROJECT_ID}/logs/compute.googleapis.com%2Ffirewall \
  AND (jsonPayload.connection.src_ip=\"${VM_ALLOW_IP}\" OR
  jsonPayload.connection.src_ip=\"${VM_DENY_IP}\")" \
  --project=${PROJECT_ID} \
  --freshness=30m \
  --limit=50 \
  --format="table(
    timestamp:label=TIMESTAMP,
    jsonPayload.connection.src_ip:label=SRC_IP,
    jsonPayload.connection.src_port:label=SRC_PORT,
    jsonPayload.connection.dest_ip:label=DEST_IP,
    jsonPayload.connection.dest_port:label=DEST_PORT,
    jsonPayload.disposition:label=ACTION,
    jsonPayload.rule_details.priority:label=PRIORITY,
    jsonPayload.load_balancer_details.forwarding_rule_name:label=FWD_RULE
  )"

```

記錄輸出內容會顯示政策強制執行的有效規則：

- 規則 2011 允許所有負載平衡器的所有 `vm-allow` 流量
- 規則 2998 會拒絕所有傳輸至負載平衡器的流量

TIMESTAMP	SRC_IP	SRC_PORT	DEST_IP	DEST_PORT	ACTION	PRIORITY
FWD_RULE						
YYYY-MM-DDTHH:MM:SS.850967068Z	10.0.0.11	48480	10.0.0.103	443	ALLOWED	2003
fr-foo-gcs						
YYYY-MM-DDTHH:MM:SS.418613380Z	10.0.0.11	37340	10.0.0.101	80	ALLOWED	2003
fr-foo-www						
YYYY-MM-DDTHH:MM:SS.213234118Z	10.0.0.12	55950	10.0.0.103	443	DENIED	2998
fr-foo-gcs						
YYYY-MM-DDTHH:MM:SS.981484412Z	10.0.0.11	41738	10.0.0.101	80	ALLOWED	2003
fr-foo-www						
YYYY-MM-DDTHH:MM:SS.189358071Z	10.0.0.12	55950	10.0.0.103	443	DENIED	2998
fr-foo-gcs						
YYYY-MM-DDTHH:MM:SS.061463883Z	10.0.0.12	55950	10.0.0.103	443	DENIED	2998
fr-foo-gcs						
YYYY-MM-DDTHH:MM:SS.965498098Z	10.0.0.12	53284	10.0.0.102	8080	DENIED	2998
fr-foo-api						

您也可以使用 Logs Explorer，在 Google Cloud 控制台中查看記錄。前往 console.cloud.google.com/logs/query 並使用標準虛擬私有雲防火牆記錄 `compute.googleapis.com/firewall`。

```
logName=projects/${PROJECT_ID}/logs/compute.googleapis.com%2Ffirewall
```

記錄部分到此結束，接下來要進行清理！

9. 清除所用資源

```
# delete client compute resources
gcloud -q compute instances delete vm-deny --zone=${ZONE_1}

gcloud -q compute instances delete vm-allow --zone=${ZONE_1}

# next
```

```
# delete load balancer resources for gcs
gcloud -q compute forwarding-rules delete fr-foo-gcs --region=${REGION_1}

gcloud -q compute target-https-proxies delete proxy-foo-gcs --region=${REGION_1}

gcloud -q compute url-maps delete ilb-foo-gcs --region=${REGION_1}

gcloud -q compute backend-services delete bes-foo-gcs --region=${REGION_1}

gcloud -q compute addresses delete vip-foo-gcs --region=${REGION_1}

# next
```

```
# delete load balancer resources for api
gcloud -q compute forwarding-rules delete fr-foo-api --region=${REGION_1}

gcloud -q compute target-http-proxies delete proxy-foo-api --region=${REGION_1}

gcloud -q compute url-maps delete ilb-foo-api --region=${REGION_1}

gcloud -q compute backend-services delete bes-foo-api --region=${REGION_1}

gcloud -q compute health-checks delete hc-foo-api --region=${REGION_1}

gcloud -q compute addresses delete vip-foo-api --region=${REGION_1}

# next
```

```
# delete load balancer resources for www
gcloud -q compute forwarding-rules delete fr-foo-www --region=${REGION_1}

gcloud -q compute target-http-proxies delete proxy-foo-www --region=${REGION_1}

gcloud -q compute url-maps delete ilb-foo-www --region=${REGION_1}

gcloud -q compute backend-services delete bes-foo-www --region=${REGION_1}

gcloud -q compute health-checks delete hc-foo-www --region=${REGION_1}

gcloud -q compute addresses delete vip-foo-www --region=${REGION_1}

# next
```

```
# delete service backend resources
gcloud -q storage rm --recursive gs://${BUCKET}

gcloud -q certificate-manager certificates delete cert-foo-gcs --location=${REGION_1}

gcloud -q compute network-endpoint-groups delete neg-psc-gcs --region=${REGION_1}

gcloud -q compute instance-groups managed delete mig-foo --region=${REGION_1}

gcloud -q compute instance-templates delete mig-template-foo --global

# next
```

```
# delete dns, nat, fw resources
gcloud -q dns record-sets delete gcs.foo.com --type=A --zone=zone-foo

gcloud -q dns record-sets delete api.foo.com --type=A --zone=zone-foo

gcloud -q dns record-sets delete www.foo.com --type=A --zone=zone-foo

gcloud -q dns managed-zones delete zone-foo

gcloud -q compute routers delete cr-nat-foo --region=${REGION_1}

gcloud -q compute network-firewall-policies associations delete \
  --firewall-policy=fw-policy-foo-${REGION_1} \
  --name=fw-policy-association-foo-${REGION_1} \
  --firewall-policy-region=${REGION_1}

gcloud -q compute network-firewall-policies delete fw-policy-foo-${REGION_1} --
region=${REGION_1}

gcloud -q network-security address-groups delete uhc-probes --location=${REGION_1}

# next
```

```
# delete network resources
gcloud -q compute networks subnets delete subnet-foo-3 --region=${REGION_1}

gcloud -q compute networks subnets delete subnet-foo-2 --region=${REGION_1}

gcloud -q compute networks subnets delete subnet-foo-1 --region=${REGION_1}

gcloud -q compute networks delete vnet-foo

# next
```

```
# delete shell variables and local files
unset PROJECT_ID REGION_1 ZONE_1 VM_ALLOW_IP VM_DENY_IP BUCKET

rm vm-server-startup.sh create_lbs.sh foo-gcs-key.pem foo-gcs-cert.pem

# end
```

步驟 10 · 約 0 分鐘

10. 結論

恭喜！您已成功設定負載平衡器的 Cloud NGFW *Essentials* ！

歡迎透過這份[意見回饋表單](#)提供任何意見、問題或修正建議

感謝您！
